



2.7. Коллекции: многообразие реализаций

Привет!

На связи шпаргалка урока 2.7. Коллекции: многообразие реализаций. В сегодняшней шпаргалке мы глубже познакомимся с функционалом коллекций, а также изучим ассоциативный массив (`map`).

Начнем с того, что умеют и множества, и списки.

Время прочтения: 30 минут.

Иерархия коллекций

Выбор необходимой коллекции

ArrayList

Как хранятся элементы

Как добавляются элементы

Как удаляются элементы

Как происходит поиск и получение элемента

Инициализация коллекции несколькими элементами

Создание неизменяемых коллекций

Добавление в коллекцию элементов другой коллекции

Очистка коллекции

Проверка коллекции на наличие в ней элементов

Удаление элементов в цикле (iterator)

Список (list)

Поиск элемента

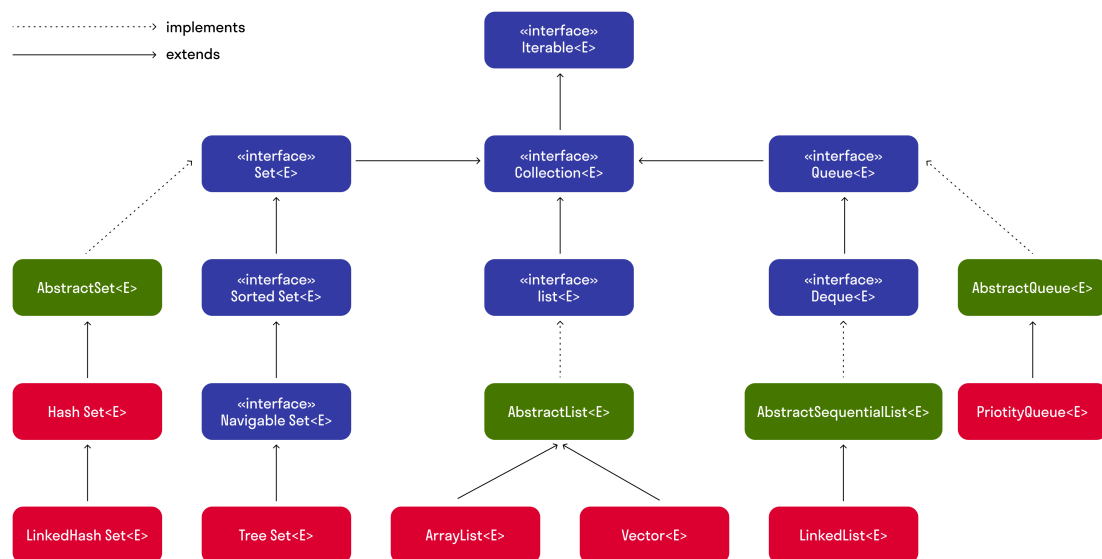
Модификация элементов

Сортировка

ListIterator

Иерархия коллекций

В Java у коллекций есть строгая иерархия. Она необходима как для полиморфизма, так и для того, чтобы все коллекции были стандартизированы, ведь их набор методов задан в интерфейсе.



Все коллекции наследуются от интерфейса `Iterable`, который позволяет итерироваться по ним с помощью цикла `foreach`, кроме `map`. Эта структура позволяет итерироваться по себе с помощью вспомогательного множества (`set`) из пар, а это множество уже, как мы видим выше, реализует `Iterable` напрямую.

Далее идет интерфейс `Collection`, который затем наследуют уже более узкие структуры. На схеме мы видим и списки, и множества, и очереди, которые представляют собой более хитрую реализацию списка.

Это отражено в классе `LinkedList`, который является и списком, и очередью одновременно.

Выбор необходимой коллекции

В Java не просто так имеется несколько реализаций каждого вида коллекций: каждая реализация коллекции имеет свои плюсы и минусы.

Возьмем, например, два списка, которые уже были рассмотрены ранее.

`ArrayList`

Плюсы:

- Быстрое обращение по индексу.
- Быстрое добавление в конец.

Минусы:

- Долгое удаление из начала или середины (нужно сдвигать «хвост», затирая удаленный элемент).
- Долгое добавление в начало или середину (нужно сдвигать «хвост»).

`LinkedList`

Плюсы:

- Очень быстрое добавление в любую часть списка (нужно просто перезаписать ссылки объекта `Node`, вставив в нужное место новый объект).
- Очень быстрое удаление (по той же причине).
- Быстрое обращение к первому или последнему элементу (ссылки на них хранятся всегда в списке).

Минусы:

- Медленный поиск (нужно идти последовательно по ссылкам).
- Медленное обращение по индексу (индексов нет, нахождение нужного «индекса» происходит путем подсчета от первого элемента к нужному).

- Долгий поиск (нужно последовательно идти по всем элементам).

Как видите, плюсы и минусы у коллекций разные.

В абсолютном большинстве ситуаций вы будете использовать `ArrayList`, но бывают ситуации, когда нужно часто добавлять или удалять элемент из начала.

В этой ситуации лучшим вариантом будет `LinkedList`, так как ему достаточно просто модифицировать два значения поля — и элемент добавлен.

В случае с `ArrayList` вам бы пришлось каждый раз двигать все имеющиеся элементы на одну ячейку вправо.

Но если вам нужно добавлять элементы в конец, `ArrayList` и `LinkedList` практически равноценны, кроме тех случаев, когда `ArrayList` приходится расширяться, пересоздавая массив.

А вот при добавлении элементов в центр ситуация не такая однозначная.

Представим, что у нас есть два списка на миллион элементов.

Один из них на основе массива, другой — на основе связанного списка.

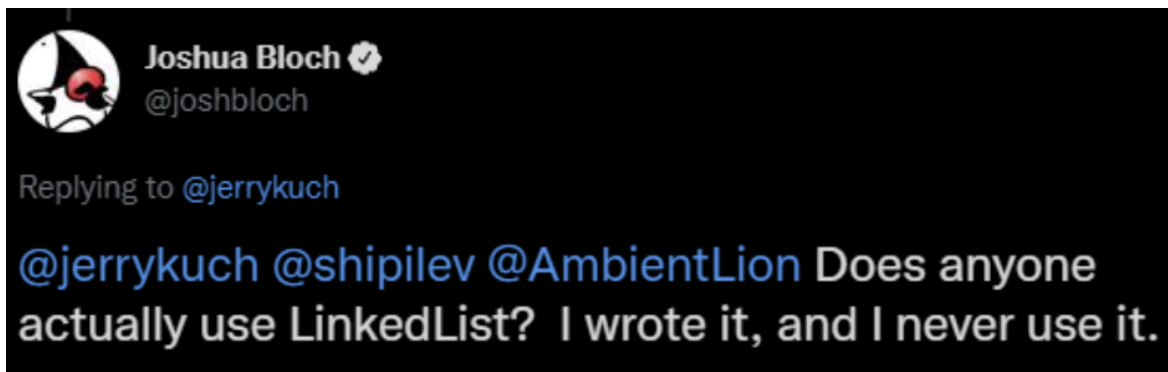
В случае, когда нам нужно положить элемент ровно в середину, победу в быстродействии, скорее всего, одержит именно `ArrayList`, так как он точно знает, что середина лежит по индексу 500_000. Для добавления ему нужно просто сдвинуть 500_000 элементов вправо, а затем положить на освободившееся место новое значение.

`LinkedList` будет проходить линейно по 500_000 элементов, считая каждый из них.

Безусловно, добавление в 500_000-ю ячейку произойдет быстрее, так как ссылки удалятся очень быстро, но вот прохождение и подсчет ячеек — дело долгое. И чем больше элементов в списке, тем дольше связанный список будет идти до нужного элемента.

▼ P. S.

`ArrayList` используется настолько часто, что даже автор реализации `LinkedList` в стандартной библиотеке Java Joshua Bloch шутит по этому поводу в твиттере. Или не шутит.



Коллекции имеют множество методов, мы разберем основные возможности коллекций по пунктам.

ArrayList

Как хранятся элементы

Внутри `ArrayList` есть некий массив объектов класса `Object`, где и хранятся ваши элементы.

- ▼ Так как все объекты в Java являются скрытыми наследниками `Object`, нет никакой проблемы хранить любые типы объектов в этом массиве без привязки к определенному типу.

```

The array buffer into which the elements of the ArrayList are stored. The capacity of the ArrayList
is the length of this array buffer. Any empty ArrayList with elementData ==
DEFAULTCAPACITY_EMPTY_ELEMENTDATA will be expanded to DEFAULT_CAPACITY when the
first element is added.
transient Object[] elementData; // non-private to simplify nested class access

```

Он даже не помечен модификатором `private`. Так как доступ должен быть предоставлен другим классам из пакета `java.util`.

▼ В момент создания `ArrayList` массив `elementData` инициализируется константой с пустым массивом, которая тоже присутствует внутри класса `ArrayList`.

```

Constructs an empty list with an initial capacity of ten.
@Contract(pure = true)
public ArrayList() { this.elementData = DEFAULTCAPACITY_EMPTY_ELEMENTDATA; }

```

```

Shared empty array instance used for default sized empty instances. We distinguish this from
EMPTY_ELEMENTDATA to know how much to inflate when first element is added.
private static final Object[] DEFAULTCAPACITY_EMPTY_ELEMENTDATA = {};

```

▼ Если вы решите вызвать конструктор `ArrayList`, где в скобках сразу будет указана стартовая размерность (`initialCapacity`), массив будет создан и переменная `elementData` будет инициализирована именно на такое количество элементов, которое вы укажете в качестве аргумента.

```

Constructs an empty list with the specified initial capacity.
Params: initialCapacity – the initial capacity of the list
Throws: IllegalArgumentException – if the specified initial capacity is negative
@Contract(pure = true)
public ArrayList( @Range(from = 0, to = java.lang.Integer.MAX_VALUE) int initialCapacity) {
    if (initialCapacity > 0) {
        this.elementData = new Object[initialCapacity];
    } else if (initialCapacity == 0) {
        this.elementData = EMPTY_ELEMENTDATA;
    } else {
        throw new IllegalArgumentException("Illegal Capacity: "+
                                         initialCapacity);
    }
}

```

Как добавляются элементы

▼ В тот момент, когда вы вызываете метод `add`, метод получает элемент неизвестного типа `E`, который вы при создании списка указываете в `<>`. В момент компиляции все `E` заменятся тем типом, что указан у вас в `<>`. Если список состоит из строк, то все `E` во время компиляции будут заменены на `String`.

```

Appends the specified element to the end of this list.
Params: e – element to be appended to this list
Returns: true (as specified by Collection.add)
public boolean add(E e) {
    modCount++;
    add( e: e, elementData: elementData, s: size);
    return true;
}

```

Когда компиляция проходит успешно, тип `E` затирается и превращается в `Object`.

На стадии исполнения ваш код совсем не знает, что лежит в списке, а работа происходит с типом `Object`.

Это процесс называется **затиранием типов** и используется он, в первую очередь, для полной совместимости со старым кодом (Java < 5), когда generics еще не было добавлено. Подробнее узнаете на отдельном уроке по generics.

Метод `add` вызывает еще один метод `add`, уже приватный.

Передаёт ему элемент, хранилище (куда нужно положить элемент) и текущий размер, который одновременно равен индексу следующей ячейки.

- ▼ Этот метод проверяет, что `size` меньше размера текущего хранилища, а затем добавляет в ячейку номер `size` элемента, затем увеличивая `size` на 1.

```
This helper method split out from add(E) to keep method bytecode size under 35 (the -XX:
MaxInlineSize default value), which helps when add(E) is called in a C1-compiled loop.
private void add(E e, @NotNull Object[] elementData, int s) {
    if (s == elementData.length)
        elementData = grow();
    elementData[s] = e;
    size = s + 1;
}
```

- ▼ Если `size` оказывается равен текущей длине `elementData`, вызывается метод `grow`.

```
@Contract(value = "->new", mutates = "this")
private Object[] grow() {
    return grow(minCapacity: size + 1);
}
```

Он делегирует выполнение другому методу `grow`, передавая ему значение `size + 1` в качестве минимального размера расширения.

- ▼ Второй метод `grow` уже занимается тем, что копирует наш массив с новым размером, который высчитывается методом `newCapacity` на основе значения `minCapacity`, присваивая его полю `elementData`. В этот момент старый массив теряет ссылку на себя (ведь `elementData` начинает ссылаться на новую копию) и помечается как мусор, а затем удаляется.

```
Increases the capacity to ensure that it can hold at least the number of elements specified by the
minimum capacity argument.
Params: minCapacity – the desired minimum capacity
Throws: OutOfMemoryError – if minCapacity is less than zero
@Contract(value = "->new", mutates = "this")
private Object[] grow(int minCapacity) {
    return elementData = Arrays.copyOf(original: elementData,
                                      newLength: newCapacity(minCapacity));
}
```

- ▼ Чтобы осуществить добавление по индексу, нужно в первую очередь проверить, что индекс попадает в текущий массив. После этого — проверить массив на необходимость расширения и вызвать метод `grow`. В итоге все элементы после указанного в аргументах индекса смещаются вправо на 1 (часть массива сдвигается на 1 ячейку вправо), а затем в клеточку по индексу добавляется элемент, увеличивая `size` на 1.

```

Inserts the specified element at the specified position in this list. Shifts the element currently at that
position (if any) and any subsequent elements to the right (adds one to their indices).
Params: index – index at which the specified element is to be inserted
        element – element to be inserted
Throws: IndexOutOfBoundsException –

public void add(int index, E element) {
    rangeCheckForAdd(index);
    modCount++;
    final int s;
    Object[] elementData;
    if ((s = size) == (elementData = this.elementData).length)
        elementData = grow();
    System.arraycopy(src: elementData, srcPos: index,
                     dest: elementData, destPos: index + 1,
                     length: s - index);
    elementData[index] = element;
    size = s + 1;
}

```

Как удаляются элементы

▼ В тот момент, когда вы вызываете метод `remove` и указываете индекс элемента в качестве параметра, первым делом проверяется, что индекс попадает в интервал от 0 до `size - 1`. Если нет, выбрасывается исключение. Затем по этому индексу получается значение и преобразовывается в тип `E`, так как изначально все элементы `elementData` являются `Object`. Это значение сохраняется в переменную для того, чтобы вернуть его в качестве результата.

```

/**
 * Removes the element at the specified position in this list.
 * Shifts any subsequent elements to the left (subtracts one from their
 * indices).
 *
 * @param index the index of the element to be removed
 * @return the element that was removed from the list
 * @throws IndexOutOfBoundsException {@inheritDoc}
 */
@Contract(mutates = "this")
public E remove(int index) {
    Objects.checkIndex(index, length: size);
    final Object[] es = elementData;

    @SuppressWarnings("unchecked") E oldValue = (E) es[index];
    fastRemove(es, index);

    return oldValue;
}

```

▼ Далее происходит вызов метода `fastRemove`, который и затирает нужный нам элемент.

```

Private remove method that skips bounds checking and does not return the value removed.
@Contract(mutates = "this,param1")
private void fastRemove(@NotNull Object[] es, int i) {
    modCount++;
    final int newSize;
    if ((newSize = size - 1) > i)
        System.arraycopy(src: es, srcPos: i + 1, dest: es, destPos: i, length: newSize - i);
    es[size = newSize] = null;
}

```



Задача метода в том, чтобы сдвинуть все элементы после нужного нам индекса на 1 влево, затерев элемент, который стоит по переданному в метод индексу.

Затем присвоить ячейке по последнему индексу до сдвига значение `null`, так как ее элемент был сдвинут на 1 шаг влево, значит, она должна освободиться.

Если вы вызываете методе `remove` не по индексу, а по объекту, то в первую очередь будет найден индекс этого элемента, а уже затем будет вызван знакомый нам ранее `fastRemove`.

Код поиска индекса из метода `remove` слишком плох и сложен, чтобы его показывать вам, но можете с ним ознакомиться на свой страх и риск в классе `ArrayList` стандартной библиотеки Java.

Как происходит поиск и получение элемента

В случае вызова метода `get` происходит простое обращение к массиву по индексу, что вы передали в качестве аргумента при вызове метода `get`.

В случае поиска индекса (`indexOf`) или проверки на существование элемента в списке (`contains`) происходит достаточно стандартный проход в цикле по всем элементам массива `elementData` и возврат необходимого индекса. Иначе, если элемент не найден, происходит возврат значения `-1`.

В случае с `contains` метод проверяет, что `indexOf` вернул не `-1` (значит, элемент в списке есть), и возвращает `true/false`.

Инициализация коллекции несколькими элементами

Часто случаются ситуации, когда нам нужно не только создать коллекцию, но и заполнить ее стартовым набором данных.

Это может делаться стандартным способом.

▼ Код тут

```
List<Integer> nums = new ArrayList<>();  
  
nums.add(1);  
nums.add(2);  
nums.add(3);
```

Этот способ многострочный и неоптимальный, ведь при добавлении данных `ArrayList` каждый раз будет проверять, хватает элементу места или нет.

С массивами мы применяли следующий способ инициализации:

```
int[] nums = {1, 2, 3};
```

Чтобы добиться чего-то похожего, нам нужно передать значения в конструктор нашей коллекции.

Например:

```
List<Integer> nums = new ArrayList<>(Arrays.asList(1, 2, 3));
```

В Java 11 уже присутствуют методы, позволяющие не использовать класс `Arrays` для инициализации списков.

Теперь вы можете писать проще, избавившись от лишнего импорта.

```
Set<Integer> nums = new HashSet(Set.of(1, 2, 3));
```

Аналогичный метод есть у интерфейса `List`.

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));
```

Именно таким способом вы можете инициализировать вашу коллекцию значениями «по умолчанию».

Создание неизменяемых коллекций

После предыдущего раздела должен возникнуть вопрос: «Зачем нам оборачивать `Set.of` или `List.of` в `HashSet` или `ArrayList` соответственно?»



Дело в том, что эти методы, как и `Arrays.asList`, создают коллекцию, которая не позволяет менять себя.

Попробуем добавить элемент в такую коллекцию.

```
List<Integer> nums = List.of(1, 2, 3);  
nums.add(4);
```

При попытке добавить или удалить элемент в список `nums` будет выброшено исключение `UnsupportedOperationException`.

Именно поэтому мы оборачиваем такие коллекции в конструктор нашей изменяемой коллекции, а затем уже работаем непосредственно с ней.

Но бывают ситуации, когда нам нужно из *изменяемой* коллекции сделать *неизменяемую*.

Например, нам нужно изменить список сотрудников из последнего домашнего задания. Работать с ним должен только определенный сервис, но вернуть пользователю список сотрудников должен контроллер.

Следовательно, геттер для этой коллекции должен быть реализован, но как запретить классам извне модифицировать наш список, ведь за его консистентность отвечает исключительно наш сервис?

В этом случае у нас есть **три** способа.



С помощью метода `List/Set.copyOf` или `Collections.copy` сделать копию нашей коллекции и вернуть ее. Другие классы смогут работать с этой коллекцией, но ее изменение не повлияет на изначальный вариант, который хранится в нашем сервисе.

```
Set<Integer> nums = new HashSet<>(Set.of(1, 2, 3));  
return Set.copyOf(nums);
```

Так как мы вернули копию, нашему множеству ничего не угрожает.

Более того, так как мы воспользовались методом интерфейса `Set`, любой внешний класс, что запросит наше множество, получит именно его неизменяемую версию.



С помощью того же класса `Collections` «обернуть» нашу коллекцию в неизменяемый вид. Примерно так же, как было при передаче одной коллекции в другую в разделе выше, только теперь мы из изменяемой делаем неизменяемую, а не наоборот.

Это делается с помощью статических методов вида `unmodifiableList/Set`.

Этот метод оборачивает вашу коллекцию в другую, методы по изменению в которой также заблокированы исключением.

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));  
return Collections.unmodifiableList(nums);
```



Вернуть абсолютно новую коллекцию со всеми элементами прошлой.

Подробнее об этом в следующем разделе.

Все варианты создают дополнительный объект, но способ с использованием метода `copyOf` более новый.

Если вам придется работать с проектами на Java, версия которой ниже 10-й, там метод `copyOf` будет отсутствовать и придется работать именно через `Collections`.

Добавление в коллекцию элементов другой коллекции

Периодически может возникнуть ситуация, когда нужно добавить в одну коллекцию все элементы из другой.



Добавить все элементы одной коллекции в другую мы можем двумя способами, и это зависит от того, создана уже новая коллекция или нет.

Чтобы создать копию коллекции, добавив туда все элементы из предыдущей, нужно в конструктор одной коллекции передать другую.

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));  
List<Integer> nums2 = new ArrayList<>(nums);
```

В этой ситуации мы получаем две одинаковые коллекции, так как все элементы `nums` были добавлены в `nums2` в момент создания этой коллекции.

Если коллекция создана ранее, нам следует воспользоваться методом `addAll`.

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));  
List<Integer> nums2 = new ArrayList<>(List.of(4, 5, 6));  
nums.addAll(nums2);
```

Теперь в коллекции `nums` присутствуют все 6 элементов, которые присутствовали в `nums` и `nums2` ранее.

У `addAll` в случае списка имеется вариант добавления элементов на определенную позицию по индексу.

Очистка коллекции

Очистить коллекции можно с помощью метода `clear`, который имеется у всех коллекций (и даже `map`).

```
Set<Integer> numSet = new HashSet<>(Set.of(1, 2, 3));  
List<Integer> numList = new ArrayList<>(numSet);  
numSet.clear();  
numList.clear();
```

После этого обе коллекции очистятся.

Проверка коллекции на наличие в ней элементов

Проверить, что коллекции не пустые, можно с помощью метода `isEmpty()`, он тоже общий для всех возможных коллекций.

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));
List<Integer> nums2 = new ArrayList<>();
boolean numsIsEmpty = nums.isEmpty(); // false
boolean nums2IsEmpty = nums2.isEmpty(); // true
```

Удаление элементов в цикле (Iterator)

Ранее упоминалось, что в цикле `forEach` удалять элементы из коллекций нельзя, так как будет выброшено исключение.



При необходимости удалять элементы можно использовать стандартный цикл `for`, но нужно помнить, что с множествами его использовать нельзя (нет индексов), и даже если индексы есть, каждое удаление будет переписывать элементы по индексам, и на этом можно ошибиться.

На помощь приходит объект, который называется **Iterator**.

Его задача — пройти по коллекции и в случае необходимости удалить нужный элемент.

Получить его можно с помощью метода `iterator()`, который есть у всех коллекций, подразумевающих возможность прохождения по ним. Т. е. в `map` он отсутствует, так как `map` не является реализацией интерфейса `Iterable`, метод которого и предоставляет нам этот самый объект, но об итерациях по `map` мы поговорим далее.

Чтобы использовать итератор, его нужно получить из коллекции.

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));
Iterator<Integer> iterator = nums.iterator();
```

Как видите, `Iterator` тоже является дженериком.

Этот объект имеет всего три метода, а именно:



next — переводит итератор на следующий элемент, возвращая его.



hasNext — проверяет, что следующий элемент есть, возвращая `true/false`.



remove — удаляет элемент, который в данный момент выбран итератором.

Попробуем удалить из упомянутой выше коллекции `nums` все нечетные элементы.

```
while (iterator.hasNext()) {
    if (iterator.next() % 2 == 1) {
        iterator.remove();
    }
}
```

После этого цикла в нашем списке `nums` останется только значение 2.

Так как при написании цикла выше IDEA вам выделит цикл желтым и предложит заменить его на определенный метод `removeIf`, мы бегло пройдем и по нему.

Метод `removeIf` выполняет абсолютно ту же работу, что и цикл выше.

Код выше с помощью этого метода будет выглядеть следующим образом:

```
nums.removeIf(e -> e % 2 == 1);
```

В строке выше есть пока неизвестная вам конструкция `e -> e % 2 == 1`.

Это называется лямбда-выражением, о нём вы узнаете по курсу позже.

Пока достаточно понимать, что есть некая переменная `e` (которая может быть названа любым именем, не только `e`), куда присваиваются по очереди значения из коллекции, затем рисуется стрелка, а затем с этим значением происходит то, что указано после стрелки.

Выглядит это примерно так:

```
if (e % 2 == 1) {  
    nums.remove(e);  
}
```

Каким способом пользоваться — решать исключительно вам, но подсказки IDEA игнорировать не стоит. Обычно она подсказывает правильно.

Список (list)

Перейдем к функционалу (методам), что уникальны для списков.

Поиск элемента

Так как список имеет индексы (или виртуальные индексы в случае связанного списка), в нём можно с помощью встроенных методов искать индекс определенного элемента.

Если индекс не найден (элемент отсутствует), то будет возвращено значение `-1`.

Чтобы найти индекс, нужно использовать метода `indexOf`.

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));  
int indexOf4 = nums.indexOf(4); // вернет -1, так как значение 4 отсутствует в списке nums  
int indexOf2 = nums.indexOf(2); // вернет 1, так как значение 2 стоит на второй позиции в списке, индексы которого начинаются с 0
```

Так как список может иметь дубли, мы сами выбираем, начать искать индекс с первого элемента или с последнего.

При необходимости начать с конца метода `indexOf` меняется на `lastIndexOf`.

▼ Послесловие

Многие из вас уже использовали метод `contains` при работе со списком сотрудников.

Вся логика работы метода `contains` заключается в том, что он вызывает `indexOf` и проверяет, что индекс не равен `-1`.

```
return indexOf(element) >= 0;
```

Вот весь код метода `contains`.

Потому использовать эти два метода вместе не требуется.

Если вам необходимо сначала проверить список на существование в нем элемента, а затем получить его индекс, лучше один раз вызвать `indexOf`, затем вручную проверить, что индекс не `-1`, а уже потом работать с этим индексом.

Модификация элементов

В списке имеется возможность с помощью метода `replaceAll` заменить все элементы.

Представим, что у вас есть список `nums`, который имеет числа от 1 до 10.

Ваша задача подразумевает, что все числа в списке нужно умножить на 20%.

Можно решить это с помощью цикла `for`.

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10));

for (int i = 0; i < nums.size(); i++) {
    int currentNum = nums.get(i);
    nums.set(i, currentNum * 2);
}
```

С помощью `replaceAll` вы можете заменить это следующим кодом.

```
nums.replaceAll(e -> e * 2);
```

Как видите, код стал короче и читабельнее.

Сортировка

Ранее нам, чтобы отсортировать массив, нужно было вызвать метод `Arrays.sort`, но данный метод позволяет отсортировать массив только в «естественном» порядке, т. е. по возрастанию.

Если бы нам пришлось сортировать массив по убыванию, пришлось бы вкрутить некоторое количество дополнительных строк, возможно, даже костылей.

Метод `sort` в списках позволяет сортировать списки в нужном порядке.

Метод в качестве параметра принимает объект, реализующий интерфейс `Comparator`, где мы можем выбрать параметры и принципы, которые будут учитываться при сортировке.

Мы не будем на данном уроке изучать реализации этого интерфейса и пока ограничимся теми вариантами, что уже реализованы.

Создадим снова список `nums`.

```
List<Integer> nums = new ArrayList<>(List.of(2, 1, 4, 3));
```

В случае, когда нам нужно сортировать его по возрастанию, напомним следующее:

```
nums.sort(Comparator.naturalOrder());
```

При необходимости сортировать список в обратном порядке в качестве параметра в метод `sort` нужно подать `reverseOrder()`;

```
nums.sort(Comparator.reverseOrder());
```

ListIterator

Данный итератор представляет собой стандартный итератор с дополнительными функциями.

Он может «идти» по коллекции не только вперед, но и назад.

Добавлены методы `previous` и `hasPrevious`, которые работают так же, как `next` и `hasNext`, но в обратную сторону.

Также были добавлены методы `nextIndex` и `previousIndex`, позволяющие получить индекс элемента, которые возвращены `next` или `previous`.